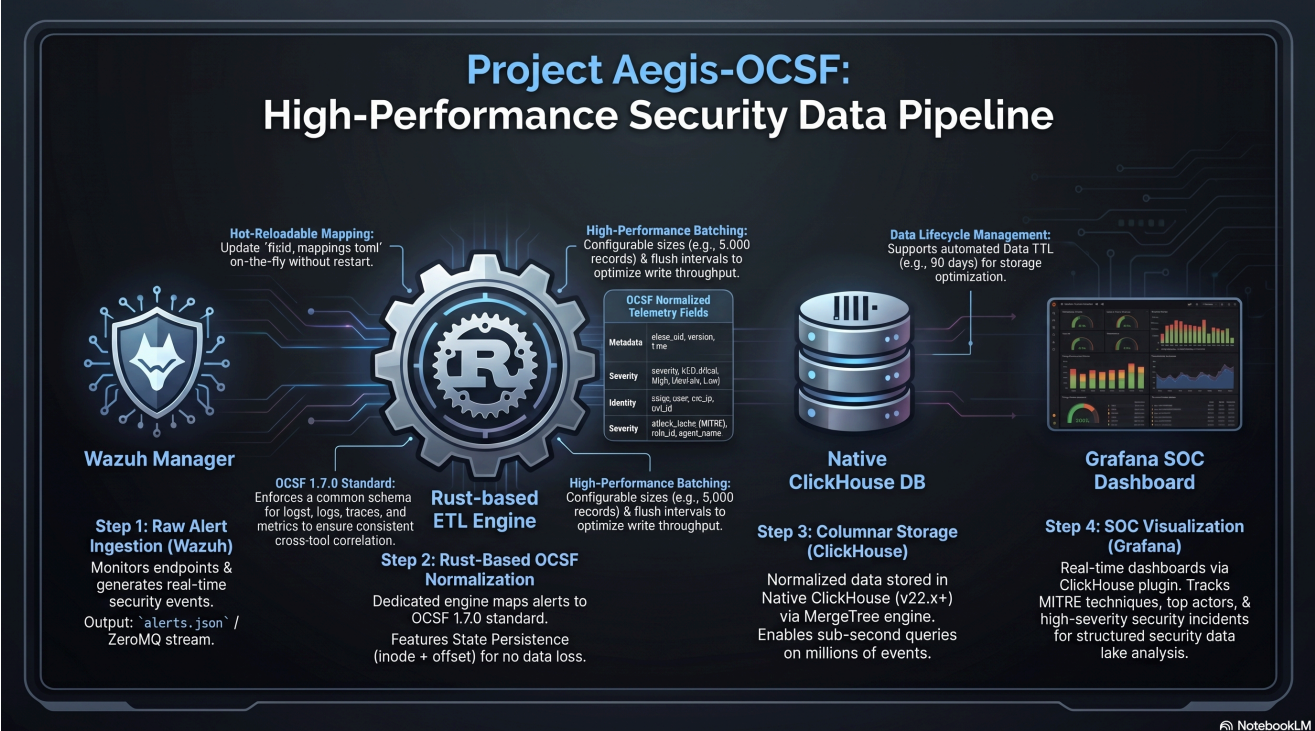


Project Aegis-OCSF Building a Scalable Security Pipeline (Wazuh to ClickHouse)



This document provides the architectural blueprint and deployment instructions for a high-performance security telemetry pipeline. We are normalizing Wazuh security alerts into the Open Cybersecurity Framework (OCSF) v1.7.0 schema—the industry standard for cross-tool normalization—and ingesting them into ClickHouse for long-term analytics and high-speed visualization.

****Architecture Overview and Prerequisites**

Project Aegis-OCSF: High-Performance Security Telemetry Pipeline

Stage 1: The Source

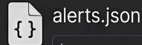
Wazuh Manager (v4.x+)



Keep Alerts and security events that foreement hoerweformuer security anat/events.

Raw Alert Generation

Captures every security event from monitored endpoints via `/var/ossec/logs/alerts/alerts.json`



```
{
  "alerts": {
    "alert_id": "alert",
    "timestamp": "2023-03-10T10:10:10.123Z",
    "source": "ossec/logs/alerts/alerts.json",
    "severity": "high",
    "class_uid": "severity",
    "actor_user": "root",
    "actor_email": "root@localhost",
    "severity": 1
  }
}
```

Stage 2: Processing (wazuh-ocsf-etl)



Rust Engine

High-Performance Rust Engine

Lightweight footprint, maximum throughput without significant CPU overhead.



OCSF 1.7.0 Normalization

Raw Wazuh alerts mapped & validated against OCSF 1.7.0 schema (class_uid, severity_id, actor_user, etc.)

Hot-Reload Mapping

Update custom mappings on-the-fly without restarting. (field_mappings.toml)



State-Persistent Ingest

Update custom mappings on-the-fly without restarting. (field_mappings). customs organized by MergeTree, ordered by class_uid and time for its observability tables (e.g., otl_logs, otl_traces) on first connection.

Stage 3: Storage (Native ClickHouse (v22.x+))



ClickHouse



MergeTree Storage Engine

Normalized data batched and written over HTTP via port 8123. Tables organized by MergeTree, ordered by class_uid and time for lightning-fast analytical queries.

High-Performance Data Warehouse

Automatically seeds required observability tables (e.g., otl_logs, otl_traces) on first connection.

High-Performance Data Warehouse

Automated Schema Management

Automatically seeds required observability tables (e.g., otl_logs, otl_traces) on first connection.

Stage 4: Visualization (Grafana SOC Dashboards)



Real-Time Insights

Grafana connects to ClickHouse as a data source for a visual frontend.



Real-Time SOC Observability

Grafana connects to ClickHouse as a data source for a visual frontend.



Actionable Insights

Analysts track key security indicators and event trends using standard SQL queries.

NotebookLM

The pipeline follows a hardened, linear data flow designed for schema consistency and operational reliability: **Wazuh Manager** (`alerts.json`) → **wazuh-ocsf-etl** (Rust-based) → **ClickHouse** → **Grafana**. The ETL tool operates in "file mode" for this deployment, monitoring the Wazuh alerts log. It maintains state using inode and offset tracking to ensure no data is lost during service restarts or log rotations.

System Prerequisites

Component	Requirement	
Operating System	Linux (x86_64) - Validated on Ubuntu 22.04 / RHEL 9	
Wazuh Manager	Version 4.x (JSON output enabled via <code><logall_json>yes</logall_json></code>)	
Database	ClickHouse 22.x+ (Reachable via HTTP)	
Visualization	Grafana (with ClickHouse Plugin installed)	
ETL Tool	wazuh-ocsf-etl (Optimized Rust binary)	

Lab Layout

A standard lab deployment utilizes the following roles:

1. **Wazuh Server:** Hosts the Wazuh Manager and the ETL binary.
2. **ClickHouse/Grafana Server:** Analytics engine and UI (Target IP: 192.168.0.104).
3. **Endpoints:** Managed nodes (Linux/Windows) running Wazuh agents to generate telemetry.

Last time i show how to install ClickHouse server and others components

<https://medium.com/@abusaeid/the-complete-a-to-z-clickhouse-deployment-guide-on-ubuntu-installation-security-and-optimization-2ed490a77c78>

Phase 1: Environment Verification

Before deploying the pipeline, you must verify that the source is producing data and the destination is reachable.

1. Verify Wazuh Alerts

The pipeline requires a valid JSON stream. Confirm the manager is actively writing to the expected path. If `alerts.json` is missing or empty, verify that your Wazuh `ossec.conf` has `<logall_json>` set to `yes` .

```
sudo ls -lh /var/ossec/logs/alerts/alerts.json
sudo tail -n 5 /var/ossec/logs/alerts/alerts.json
```

2. Verify ClickHouse Reachability

The ETL tool writes to ClickHouse over the HTTP interface (port 8123). Test connectivity from the Wazuh Server:

```
curl http://192.168.0.104:8123/ping
```

Expected Output: Ok.

3. Verify Listening Ports

On the ClickHouse server, ensure the service is bound to external interfaces (not just 127.0.0.1):

```
sudo ss -lntp | grep -E '8123|9000'
```

Phase 2: Installation of the wazuh-ocsf-etl Binary

For lab environments, the release-based installation is the most efficient path.

1. **Download the Binary and Official Script:**
2. **Execute the Installation:**

Architectural Impact: This script moves the binary to `/usr/local/bin`, creates a dedicated `wazuh-ocsf` service account, establishes the `/opt/wazuh-ocsf` directory structure, and registers the `systemd` unit.

Step-by-Step Installation

1. Download Binary

```
wget https://github.com/yash22091/wazuh-ocsf-etl/releases/download/latest/wazuh-ocsf-etl  
chmod +x wazuh-ocsf-etl
```

2. Run Installer

```
wget -q https://github.com/yash22091/wazuh-ocsf-etl/releases/download/latest/install.sh  
chmod +x install.sh  
sudo ./install.sh wazuh-ocsf-etl
```



```

Vaults SFTP 192.168.0.104
[ (root@kali) - [/home/fancybear] ]
# ls
'arduino-ide_2.3.8_Linux_64bit(1).AppImage' Desktop grafana Public Videos WAZUH-OCSF-ETL
arduino-ide_2.3.8_Linux_64bit.AppImage Documents Music Sigma vmware WAZUH-OCSF-ETL-etl
ClickHouse Downloads OS Templates Threat_Hunting wazuh
Code_post Fleet Pictures
[ (root@kali) - [/home/fancybear] ]
# cd WAZUH-OCSF-ETL
[ (root@kali) - [/home/fancybear/WAZUH-OCSF-ETL] ]
# LS
LS: command not found
[ (root@kali) - [/home/fancybear/WAZUH-OCSF-ETL] ]
# ls
install.sh wazuh-ocsf-etl
[ (root@kali) - [/home/fancybear/WAZUH-OCSF-ETL] ]
# ls -lah
total 4.3M
drwxr-xr-x 2 root root 4.0K Apr 20 02:46 .
drwx----- 41 fancybear fancybear 4.0K Apr 24 10:29 ..
-rwxr-xr-x 1 root root 25K Mar 19 11:01 install.sh
-rwxr-xr-x 1 root root 4.2M Apr 20 02:46 wazuh-ocsf-etl
[ (root@kali) - [/home/fancybear/WAZUH-OCSF-ETL] ]
# ./install.sh wazuh-ocsf-etl

— Step 1 – Install binary —
[OK] Binary installed → /usr/local/bin/wazuh-ocsf-etl
[INFO] Binary size: 4.3M

— Step 2 – Create system user —
[OK] System user 'wazuh-ocsf' already exists – skipping

— Step 3 – Create directory layout under /opt —
[OK] Directories created: /opt/wazuh-ocsf/{config,state}

— Step 4 – Write .env configuration —
[WARN] .env already exists → skipping (edit manually: /opt/wazuh-ocsf/.env)

— Step 5 – Deploy field_mappings.toml —
[OK] field_mappings.toml already exists – skipping

— Step 6 – Set file permissions —
[OK] Ownership: wazuh-ocsf:wazuh-ocsf → /opt/wazuh-ocsf
[OK] Added 'wazuh-ocsf' to the 'wazuh' group (alerts.json access)

— Step 7 – Install systemd service unit —
[OK] Service unit written → /etc/systemd/system/wazuh-ocsf-etl.service

— Step 8 – Reload systemd and enable service —
[OK] Service enabled (starts on boot)

=====
Installation complete!
=====

Binary      : /usr/local/bin/wazuh-ocsf-etl
Install dir : /opt/wazuh-ocsf
Config      : /opt/wazuh-ocsf/.env
Field map   : /opt/wazuh-ocsf/config/field_mappings.toml
State dir   : /opt/wazuh-ocsf/state/
Service unit : /etc/systemd/system/wazuh-ocsf-etl.service
Service user : wazuh-ocsf

Next steps:
1. Edit the configuration file:
   nano /opt/wazuh-ocsf/.env

2. Start the service:
   systemctl start wazuh-ocsf-etl

3. Watch live logs:
   journalctl -u wazuh-ocsf-etl -f

4. Check service status:
   systemctl status wazuh-ocsf-etl

To uninstall: sudo ./install.sh --uninstall

[ (root@kali) - [/home/fancybear/WAZUH-OCSF-ETL] ]
#
```

This will:

- Install binary to /usr/local/bin
- Create system user
- Setup /opt/wazuh-ocsf directory
- Create config + state files
- Register systemd service

3. Configure

nano /opt/wazuh-ocsf/.env

Phase 3: Configuration of the Environment (.env)

Edit the environment file at /opt/wazuh-ocsf/.env . We will configure the tool for "file mode" with historical ingestion and proper buffer management.

Configuration Template

```
CLICKHOUSE_URL=http://192.168.0.104:8123
CLICKHOUSE_DATABASE=wazuh_ocsf
CLICKHOUSE_USER=default
CLICKHOUSE_PASSWORD=

INPUT_MODE=file
ALERTS_FILE=/var/ossec/logs/alerts/alerts.json

STATE_FILE=/opt/wazuh-ocsf/state/alerts.pos
UNMAPPED_FIELDS_FILE=/opt/wazuh-ocsf/state/unmapped_fields.json
FIELD_MAPPINGS_FILE=/opt/wazuh-ocsf/config/field_mappings.toml

# Set to false for historical backfill on first run
SEEK_TO_END_ON_FIRST_RUN=false

BATCH_SIZE=5000
FLUSH_INTERVAL_SECS=5
CHANNEL_CAP=50000
DATA_TTL_DAYS=90
OCSF_VALIDATE=true
RUST_LOG=info
```

Architect's Note: We set `CHANNEL_CAP=50000` to prevent event drops during traffic spikes and `DATA_TTL_DAYS=90` to automatically manage disk pressure. Setting `SEEK_TO_END_ON_FIRST_RUN=false` is mandatory for the first run to ensure you ingest existing alerts for immediate validation.

```
GNU nano 8.7.1 /opt/wazuh-ocsf/.env
#
# wazuh-ocsf-etl - Runtime configuration
# Edit this file, then run: systemctl restart wazuh-ocsf-etl
#
# --- ClickHouse connection ---
CLICKHOUSE_URL=http://192.168.0.104:8123
CLICKHOUSE_DATABASE=wazuh-ocsf
CLICKHOUSE_USER=default
CLICKHOUSE_PASSWORD=admin123
# --- Input source ---
# file - read alerts.json from disk (default)
# zeromq - subscribe to wazuh-analysisd ZeroMQ PUB socket (no disk I/O)
INPUT_MODE=file
# Path to Wazuh alerts JSON (FILE mode only)
ALERTS_FILE=/var/ossec/logs/alerts/alerts.json
# ZeroMQ URI (ZEROMQ mode only)
# ZEROMQ_URI=tcp://localhost:11111
# --- State / config ---
STATE_FILE=/opt/wazuh-ocsf/state/alerts.pos
UNMAPPED_FIELDS_FILE=/opt/wazuh-ocsf/state/unmapped_fields.json
FIELD_MAPPINGS_FILE=/opt/wazuh-ocsf/config/field_mappings.toml
# --- First-run behaviour ---
# true = start from current end of file (skip historical data)
# false = process entire alerts.json from the beginning
SEEK_TO_END_ON_FIRST_RUN=true
# --- Throughput tuning ---
BATCH_SIZE=5000
FLUSH_INTERVAL_SECS=5
CHANNEL_CAP=50000
# --- Data retention ---
# Delete rows older than N days (leave empty to keep forever)
DATA_TTL_DAYS=90
# --- Raw data storage ---
# true = store the full raw Wazuh alert JSON in raw_data (default)
# false = write empty string - saves 40-70% table size, no data loss
STORE_RAW_DATA=true
# --- OCSF schema validation ---
# Set false during load testing to skip per-event validation
OCSF_VALIDATE=true
# --- Logging ---
# Levels: error | warn | info | debug | trace
RUST_LOG=info
# --- Special locations (optional) ---
# Comma-separated location names routed to shared tables instead
# of per-agent tables. Example:
# SPECIAL_LOCATIONS=aws_cloudtrail,okta,azure_ad
```

4. Start Service

systemctl start wazuh-ocsf-etl

systemctl status wazuh-ocsf-etl

journalctl -u wazuh-ocsf-etl -f

```

Vaults SFTP 192.168.0.104
(root@kali)-[/home/fancybear/MAZUH-OCSF-ETL]
# systemctl start wazuh-ocsf-etl
systemctl status wazuh-ocsf-etl
journalctl -u wazuh-ocsf-etl -f
Press enter or click to view image in full size

• wazuh-ocsf-etl.service - Wazuh → OCSF → ClickHouse ETL pipeline
  Loaded: loaded (/etc/systemd/system/wazuh-ocsf-etl.service; enabled; preset: disabled)
  Active: active (running) since Fri 2026-04-24 10:32:45 PDT; 11s ago
  Invocation: 1fb22d42604442d0850057eae89fb51f
  Docs: https://github.com/yash22091/wazuh-ocsf-etl
  Main PID: 22562 (wazuh-ocsf-etl)
  Tasks: 15 (limit: 18171)
  Memory: 2.1M (max: 512M, available: 509.8M, peak: 3.8M)
  CPU: 103ms
  CGroup: /system.slice/wazuh-ocsf-etl.service
          └─22562 /usr/local/bin/wazuh-ocsf-etl

Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602838Z INFO special_locations: []
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602842Z INFO data_ttl_days : Some(90)
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602850Z INFO ocsf_version : 1.7.0
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602853Z INFO custom_mappings : 0 rule(s)
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602856Z INFO ocsf_validate : true (set OCSF_VALIDATE=false to>
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602860Z INFO unmapped report : /opt/wazuh-ocsf/state/unmapped_f>
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602916Z INFO unmapped_fields: previous report archived → /opt/wazu>
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.603365Z INFO Tailing /var/ossec/logs/alerts/alerts.json from offse>
Apr 24 10:32:50 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:50.739754Z INFO table ready: wazuh_ocsf.ocsf_kali
Apr 24 10:32:50 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:50.760361Z INFO flush ok rows=10 table=wazuh_ocsf.ocsf_kali

(root@kali)-[/home/fancybear/MAZUH-OCSF-ETL]
# journalctl -u wazuh-ocsf-etl -f
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602838Z INFO special_locations: []
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602842Z INFO data_ttl_days : Some(90)
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602850Z INFO ocsf_version : 1.7.0
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602853Z INFO custom_mappings : 0 rule(s)
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602856Z INFO ocsf_validate : true (set OCSF_VALIDATE=false to
disable)
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602860Z INFO unmapped report : /opt/wazuh-ocsf/state/unmapped_fi
elds.json
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.602916Z INFO unmapped_fields: previous report archived → /opt/wazuh
-ocsf/state/unmapped_fields.20260424T173245Z.bak
Apr 24 10:32:45 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:45.603365Z INFO Tailing /var/ossec/logs/alerts/alerts.json from offset
=283495 (inode=29756206)
Apr 24 10:32:50 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:50.739754Z INFO table ready: wazuh_ocsf.ocsf_kali
Apr 24 10:32:50 kali wazuh-ocsf-etl[22562]: 2026-04-24T17:32:50.760361Z INFO flush ok rows=10 table=wazuh_ocsf.ocsf_kali
```

Phase 4: Permission Management and Access Fixes

The service account **must** have read access to the Wazuh logs. Failure to set this up correctly is the most common cause of "silent" ingestion failures.

1. **Stop the Service First:**
2. **Test Access:** Attempt to read the file as the service user:
3. **Resolution (Security Best Practice):** Do not do something reckless like making the whole log directory world-readable (777). Instead, add the service user to the wazuh group or set a specific ACL:

Phase 5: Service Management and Log Monitoring

```
sudo systemctl enable wazuh-ocsf-etl
sudo systemctl start wazuh-ocsf-etl
```

Real-Time Monitoring

Monitor the ingestion flow via `journalctl`. You are looking for successful parsing and batch insertions:

```
sudo journalctl -u wazuh-ocsf-etl -f
```

```
Linux kali 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Apr 22 11:35:05 2026 from 192.168.0.102
(fancybear@kali)~$ sudo su
[sudo] password for fancybear:
(fancybear@kali)~$ sudo journalctl -u wazuh-ocsf-etl -f

Apr 22 11:38:59 kali wazuh-ocsf-etl[962]: 2026-04-22T18:38:59.073338Z INFO flush ok rows=3 table=wazuh_ocsf.ocsf_kali
Apr 22 11:39:54 kali wazuh-ocsf-etl[962]: 2026-04-22T18:39:54.071172Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 11:39:59 kali wazuh-ocsf-etl[962]: 2026-04-22T18:39:59.073479Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
Apr 22 11:40:14 kali wazuh-ocsf-etl[962]: 2026-04-22T18:40:14.072832Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 11:40:24 kali wazuh-ocsf-etl[962]: 2026-04-22T18:40:24.072238Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
Apr 22 11:40:44 kali wazuh-ocsf-etl[962]: 2026-04-22T18:40:44.072250Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 11:40:49 kali wazuh-ocsf-etl[962]: 2026-04-22T18:40:49.072415Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
Apr 22 11:44:14 kali wazuh-ocsf-etl[962]: 2026-04-22T18:44:14.071899Z INFO flush ok rows=6 table=wazuh_ocsf.ocsf_kali
Apr 22 11:44:19 kali wazuh-ocsf-etl[962]: 2026-04-22T18:44:19.071720Z INFO flush ok rows=7 table=wazuh_ocsf.ocsf_kali
Apr 22 11:49:24 kali wazuh-ocsf-etl[962]: 2026-04-22T18:49:24.072703Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 11:49:34 kali wazuh-ocsf-etl[962]: 2026-04-22T18:49:34.075092Z INFO flush ok rows=5 table=wazuh_ocsf.ocsf_kali
Apr 22 11:52:04 kali wazuh-ocsf-etl[962]: 2026-04-22T18:52:04.071789Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 11:52:09 kali wazuh-ocsf-etl[962]: 2026-04-22T18:52:09.072947Z INFO flush ok rows=3 table=wazuh_ocsf.ocsf_kali
Apr 22 11:56:49 kali wazuh-ocsf-etl[962]: 2026-04-22T18:56:49.072843Z INFO flush ok rows=10 table=wazuh_ocsf.ocsf_kali
Apr 22 11:57:59 kali wazuh-ocsf-etl[962]: 2026-04-22T18:57:59.071092Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
Apr 22 11:58:09 kali wazuh-ocsf-etl[962]: 2026-04-22T18:58:09.068925Z INFO flush ok rows=3 table=wazuh_ocsf.ocsf_kali
Apr 22 11:58:14 kali wazuh-ocsf-etl[962]: 2026-04-22T18:58:14.071087Z INFO flush ok rows=3 table=wazuh_ocsf.ocsf_kali
Apr 22 11:58:19 kali wazuh-ocsf-etl[962]: 2026-04-22T18:58:19.070142Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 12:02:19 kali wazuh-ocsf-etl[962]: 2026-04-22T19:02:19.068002Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
Apr 22 12:03:54 kali wazuh-ocsf-etl[962]: 2026-04-22T19:03:54.073346Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 12:04:29 kali wazuh-ocsf-etl[962]: 2026-04-22T19:04:29.154442Z INFO table ready: wazuh_ocsf.ocsf_win11
Apr 22 12:04:29 kali wazuh-ocsf-etl[962]: 2026-04-22T19:04:29.224434Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_win11
Apr 22 12:04:34 kali wazuh-ocsf-etl[962]: 2026-04-22T19:04:34.131856Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_win11
Apr 22 12:05:44 kali wazuh-ocsf-etl[962]: 2026-04-22T19:05:44.074084Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
Apr 22 12:05:49 kali wazuh-ocsf-etl[962]: 2026-04-22T19:05:49.071950Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_win11
Apr 22 12:05:54 kali wazuh-ocsf-etl[962]: 2026-04-22T19:05:54.078684Z INFO flush ok rows=78 table=wazuh_ocsf.ocsf_win11
Apr 22 12:05:59 kali wazuh-ocsf-etl[962]: 2026-04-22T19:05:59.086621Z INFO flush ok rows=403 table=wazuh_ocsf.ocsf_win11
Apr 22 12:06:04 kali wazuh-ocsf-etl[962]: 2026-04-22T19:06:04.068856Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_win11
Apr 22 12:06:09 kali wazuh-ocsf-etl[962]: 2026-04-22T19:06:09.071126Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_win11
Apr 22 12:08:49 kali wazuh-ocsf-etl[962]: 2026-04-22T19:08:49.072438Z INFO flush ok rows=2 table=wazuh_ocsf.ocsf_kali
Apr 22 12:08:59 kali wazuh-ocsf-etl[962]: 2026-04-22T19:08:59.069839Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
Apr 22 12:09:09 kali wazuh-ocsf-etl[962]: 2026-04-22T19:09:09.067779Z INFO flush ok rows=3 table=wazuh_ocsf.ocsf_kali
Apr 22 12:09:24 kali wazuh-ocsf-etl[962]: 2026-04-22T19:09:24.071126Z INFO flush ok rows=9 table=wazuh_ocsf.ocsf_kali
Apr 22 12:13:29 kali wazuh-ocsf-etl[962]: 2026-04-22T19:13:29.072094Z INFO flush ok rows=1 table=wazuh_ocsf.ocsf_kali
```

ClickHouse Table Schema

Example table created automatically:

```
CREATE TABLE ocsf_agent
(
  class_uid UInt16,
  time UInt32,
  severity_id UInt8,
  src_ip String,
  dst_ip String,
  actor_user String,
  event_data String,
  raw_data String
```

```
)  
ENGINE = MergeTree  
ORDER BY (class_uid, time);
```

Phase 6: Final Data Validation (ClickHouse & Grafana)

ClickHouse Schema Verification

The ETL tool auto-creates the schema using the MergeTree engine, optimized for time-series security data and ordered by (class_uid, time).

Connect to your database to verify:

```
SHOW DATABASES;  
USE wazuh_ocsf;  
SHOW TABLES;  
  
-- Inspect the schema and row counts  
SELECT table, total_rows FROM system.tables WHERE database = 'wazuh_ocsf';  
SELECT Timestamp, ServiceName, SeverityText, Body FROM ocsf_agent LIMIT 10;
```

Note: If ocsf_agent is not present, use the results of SHOW TABLES to identify the specific agent or location tables created by your configuration.

```

Vaults SFTP 192.168.0.104
(root@kali)-[/home/fancybear]
# clickhouse-client
ClickHouse client version 26.3.9.8 (official build).
Connecting to localhost:9000 as user default.
Password for user (default):
Connecting to localhost:9000 as user default.
Connected to ClickHouse server version 26.3.9.

Warnings:
* Linux transparent hugepages are set to "always". Check /sys/kernel/mm/transparent_hugepage/enabled
* Delay accounting is not enabled, OSIOWaitMicroseconds will not be gathered. You can enable it using `sudo sh -c 'echo 1 > /proc/s
ys/kernel/task_delayacct'` or by using sysctl.

kali.fancybear :) SHOW TABLES;

SHOW TABLES

Query id: c085247c-31ea-45f0-b909-2a8447260fdd

1. | name |
   | ocsf_agent |

1 row in set. Elapsed: 0.003 sec.

kali.fancybear :) SHOW DATABASES;

SHOW DATABASES

Query id: 749a793c-fdcd-4425-b38c-07503995f1f0

1. | name |
2. | INFORMATION_SCHEMA |
3. | default |
4. | information_schema |
5. | system |
6. | wazuh_ocsf |

5 rows in set. Elapsed: 0.002 sec.

kali.fancybear :) ^C
kali.fancybear :) USE wazuh_ocsf;
SHOW TABLES;

USE wazuh_ocsf

Query id: d32bc351-a45f-4a73-abc0-6a9e26137fe0

Ok.

0 rows in set. Elapsed: 0.002 sec.

SHOW TABLES

Query id: ce74e8f1-57ac-4f3b-8fbb-31b4bc8c946e

1. | name |
2. | ocsf_kali |
3. | ocsf_win11 |

2 rows in set. Elapsed: 0.003 sec.

kali.fancybear :) DESCRIBE wazuh_ocsf.ocsf_win11;

DESCRIBE TABLE wazuh_ocsf.ocsf_win11

Query id: 6c34b72f-e083-4455-8b46-ff2e600474de

1. | name | type | default_type | default_expression | comment | codec_expression | ttl_expression |
2. | time | DateTime | | | | Delta(4), ZSTD(1) |
3. | time_dt | String | | | | ZSTD(3) |
4. | ocsf_version | LowCardinality(String) | | | | ZSTD(1) |
5. | class_uid | UInt32 | | | | ZSTD(1) |
6. | class_name | LowCardinality(String) | | | | ZSTD(1) |
7. | category_uid | UInt32 | | | | ZSTD(1) |
8. | category_name | LowCardinality(String) | | | | ZSTD(1) |
9. | severity_id | UInt8 | | | | ZSTD(1) |
10. | severity | LowCardinality(String) | | | | ZSTD(1) |
11. | activity_id | UInt8 | | | | ZSTD(1) |
```

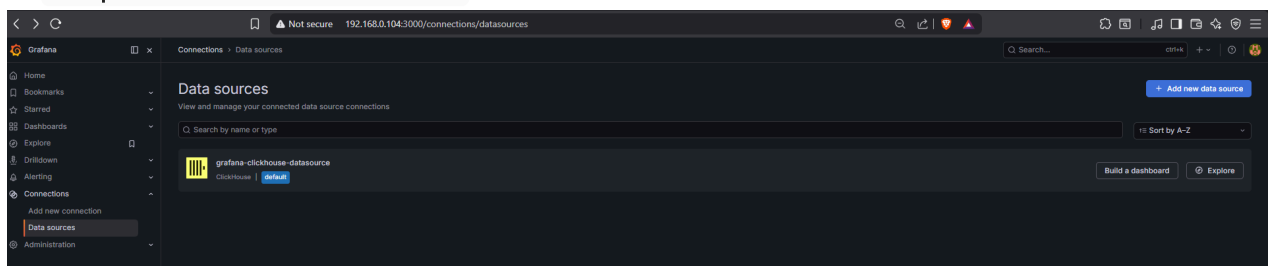
11.	activity_name	LowCardinality(String)				ZSTD(1)	
12.	type_uid	UInt32				ZSTD(1)	
13.	status_id	UInt8				ZSTD(1)	
14.	confidence_id	UInt8				ZSTD(1)	
15.	status	LowCardinality(String)				ZSTD(1)	

And also when using ClickHouse GUI

#	name	type	default_type	default_expression	comment	codec_expression	ttl_expression
1	time	DateTime				Delta(4), ZSTD(1)	
2	time_dt	String				ZSTD(3)	
3	ocsf_version	LowCardinality(String)				ZSTD(1)	
4	class_uid	UInt32				ZSTD(1)	
5	class_name	LowCardinality(String)				ZSTD(1)	
6	category_uid	UInt32				ZSTD(1)	
7	category_name	LowCardinality(String)				ZSTD(1)	
8	severity_id	UInt8				ZSTD(1)	
9	severity	LowCardinality(String)				ZSTD(1)	
10	activity_id	UInt8				ZSTD(1)	
11	activity_name	LowCardinality(String)				ZSTD(1)	
12	type_uid	UInt32				ZSTD(1)	
13	status_id	UInt8				ZSTD(1)	
14	confidence_id	UInt8				ZSTD(1)	
15	status	LowCardinality(String)				ZSTD(1)	
16	action	LowCardinality(String)				ZSTD(1)	
17	device_uid	String				ZSTD(3)	
18	device_name	LowCardinality(String)				ZSTD(1)	
19	device_ip	String				ZSTD(1)	
20	src_ip	String				ZSTD(3)	
21	dst_ip	String				ZSTD(3)	
22	src_port	UInt16				Delta(2), ZSTD(1)	
23	dst_port	UInt16				Delta(2), ZSTD(1)	
24	nat_src_ip	String				ZSTD(3)	
25	nat_dst_ip	String				ZSTD(3)	
26	nat_src_port	UInt16				Delta(2), ZSTD(1)	
27	nat_dst_port	UInt16				Delta(2), ZSTD(1)	
28	network_protocol	LowCardinality(String)				ZSTD(1)	
29	bytes_in	UInt64				Delta(8), ZSTD(1)	
30	bytes_out	UInt64				Delta(8), ZSTD(1)	
31	actor_user	String				ZSTD(3)	
32	target_user	String				ZSTD(3)	

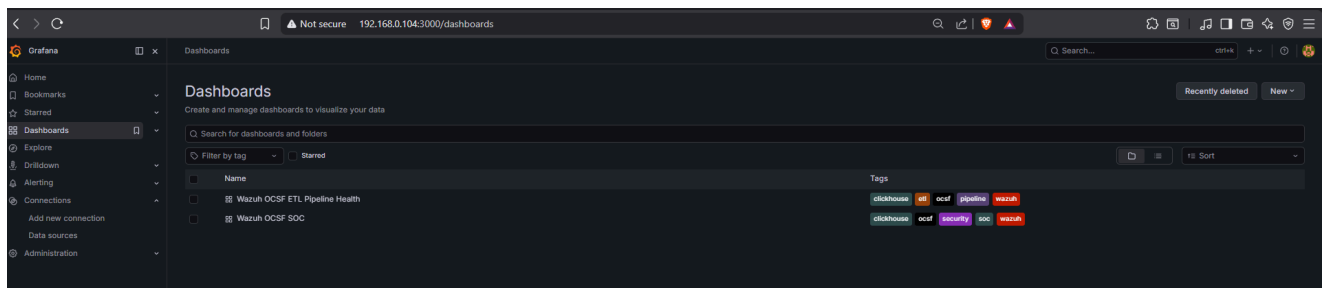
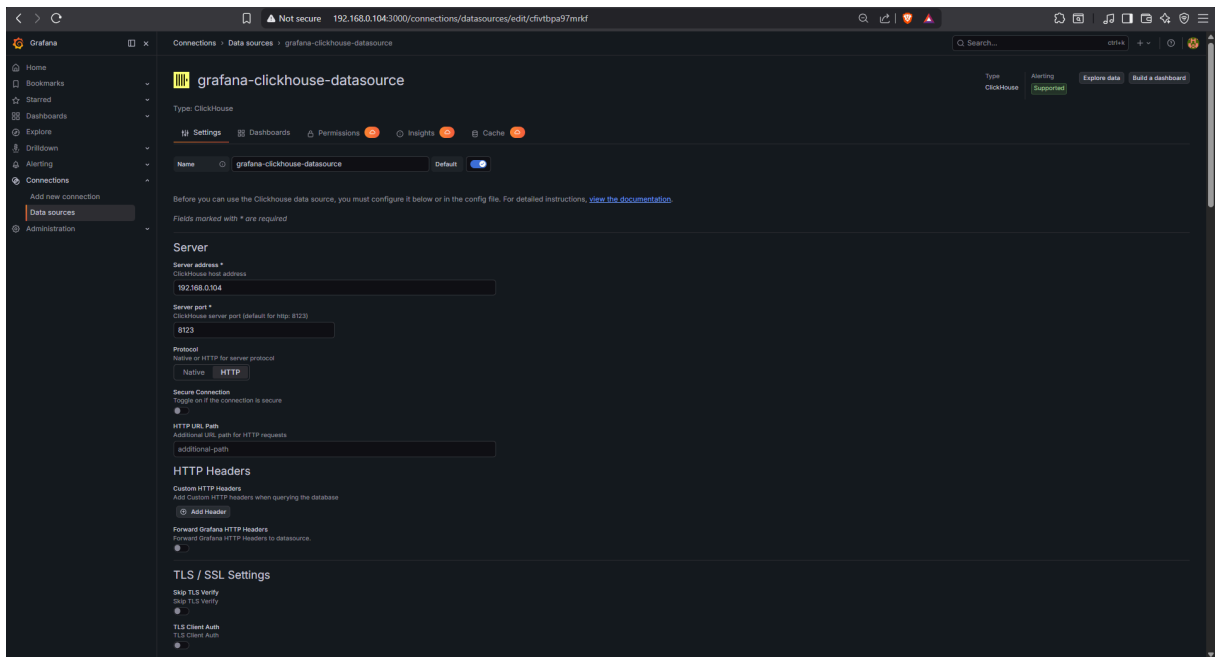
Grafana Integration

1. **Add Datasource:** Install the ClickHouse plugin and point the URL to `http://192.168.0.104:8123`.



2. **Sample Queries for Dashboards:**

- **Events Over Time:** `SELECT toStartOfMinute(time) as t, count() FROM ocsf_agent GROUP BY t ORDER BY t`
- **Top Source IPs:** `SELECT src_ip, count() FROM ocsf_agent GROUP BY src_ip ORDER BY count() DESC LIMIT 10`
- **Alerts by Severity:** `SELECT severity_id, count() FROM ocsf_agent GROUP BY severity_id`



Visualization with Grafana

Grafana is used for dashboards:

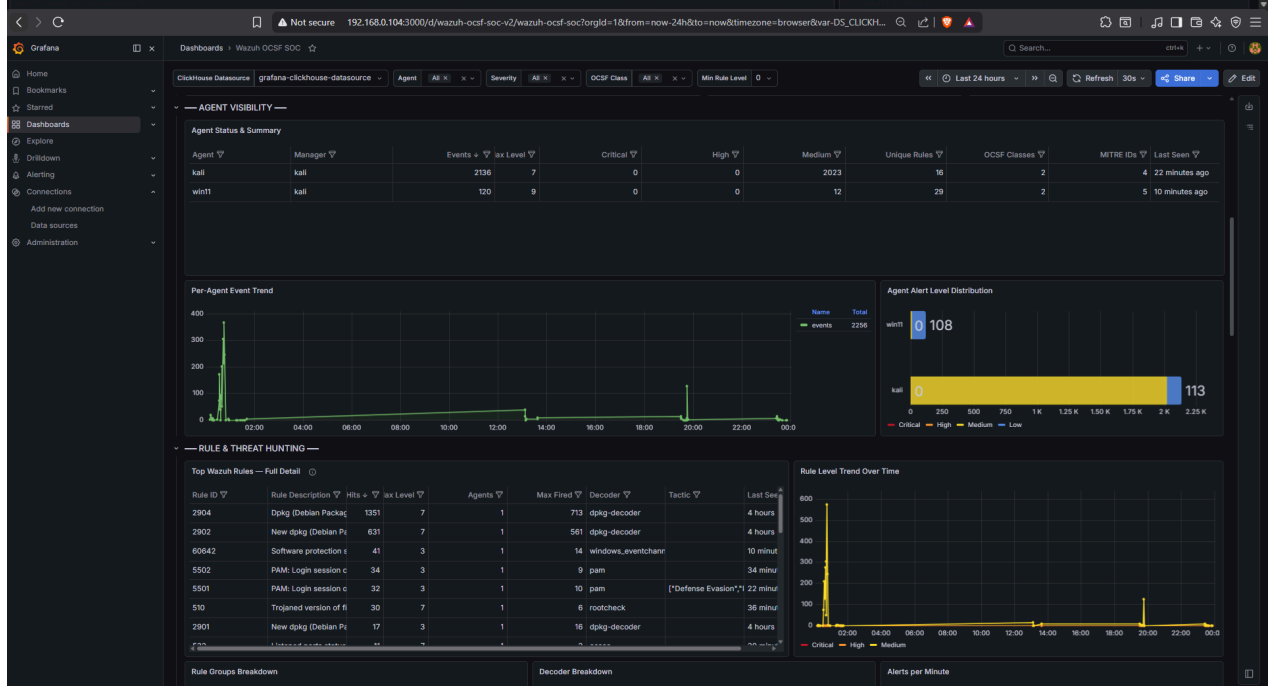
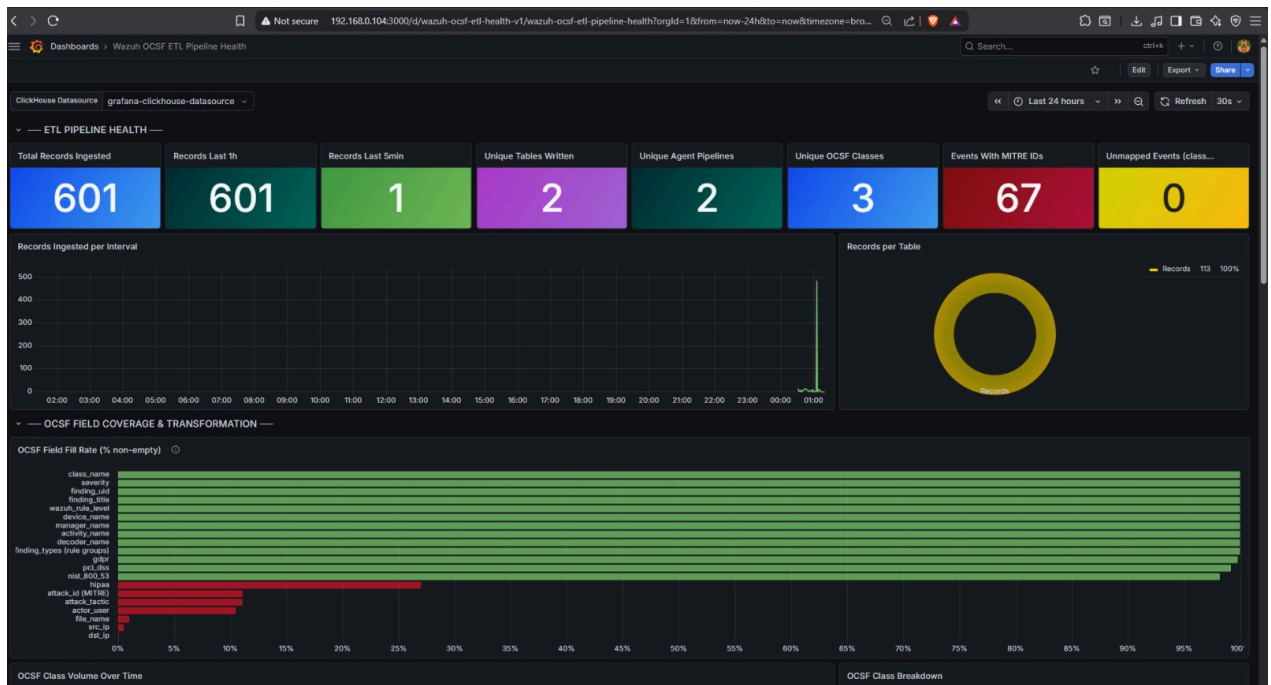
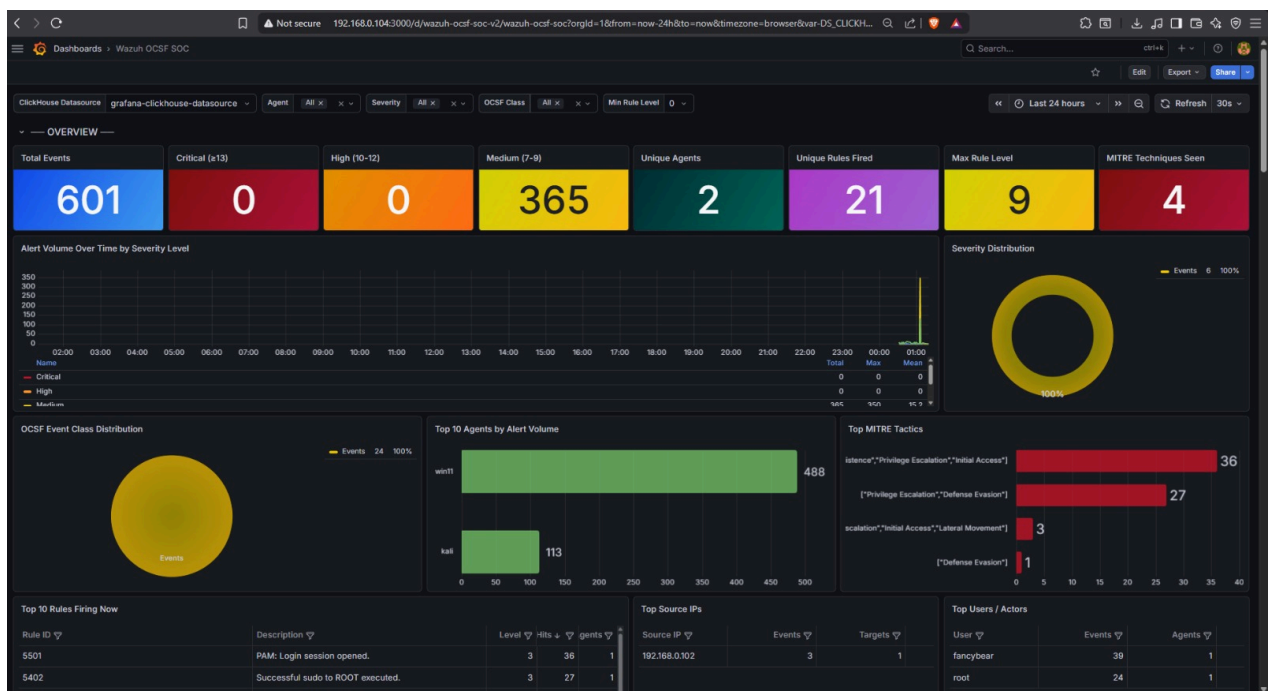
- Connects directly to ClickHouse
- Displays real-time alerts

Supports filtering by:

- severity
- class_uid
- src_ip
- user

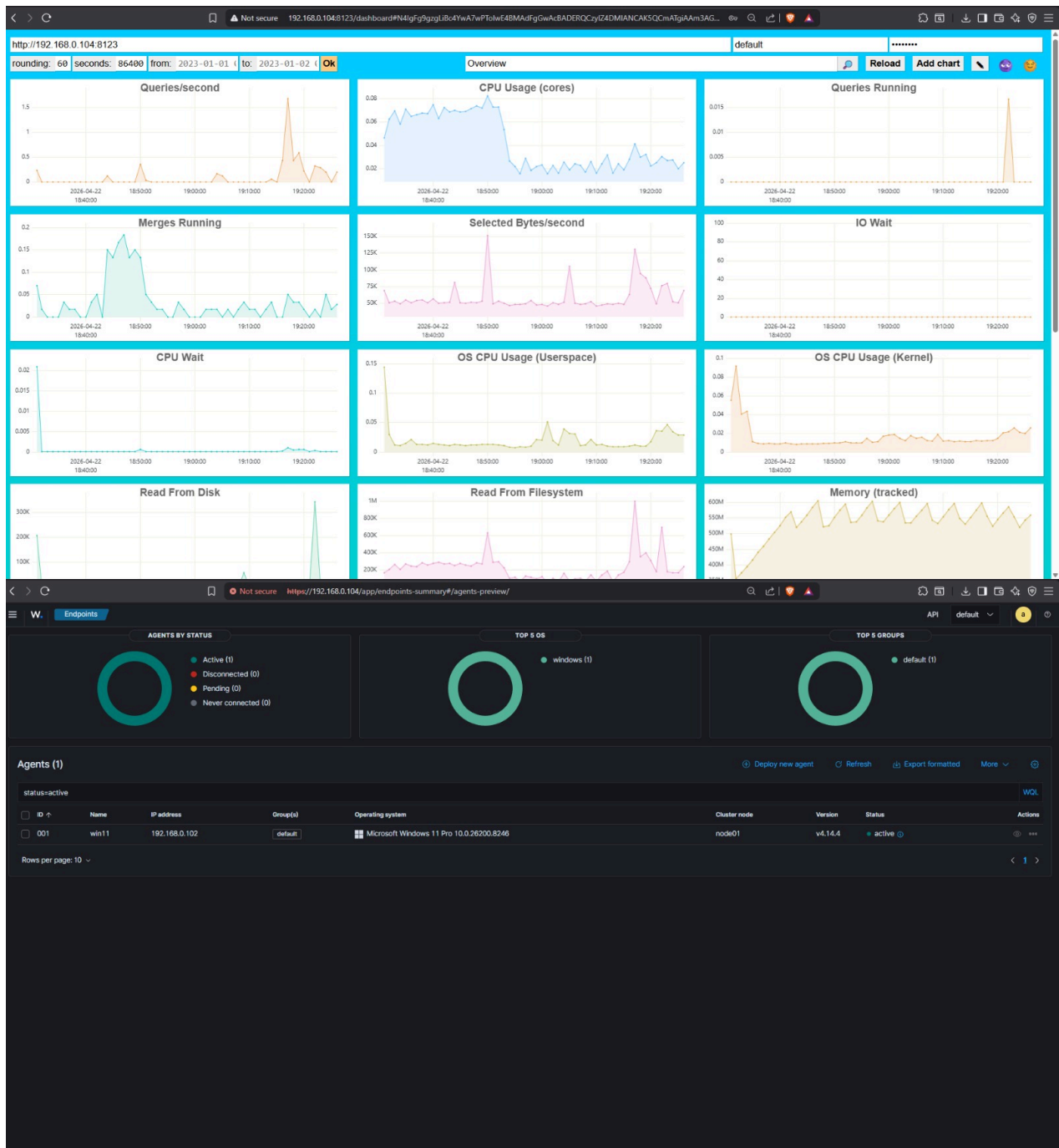
Example dashboards

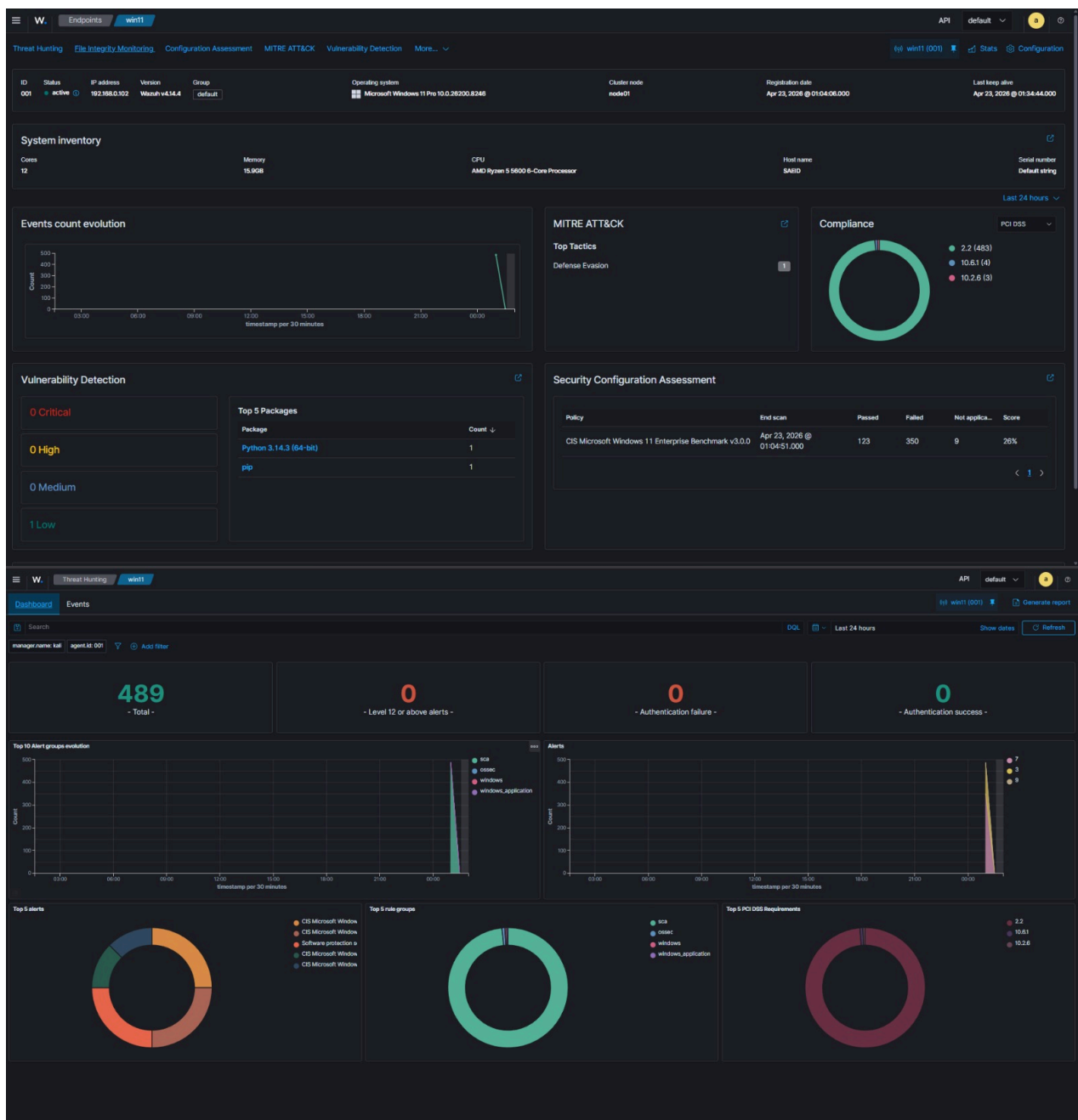
- Alert trends over time



The screenshot displays a Grafana dashboard with a dark theme. The top navigation bar includes the Grafana logo, a search bar, and user information. The left sidebar contains a menu with options like Home, Bookmarks, Starred, Dashboards, Explore, Drilldown, Alerting, Connections, Add new connection, Data sources, and Administration. The main content area is divided into several panels:

- ClickHouse Datasource**: A header bar for the data source, showing filters for Agent, All, Severity, All, OCSF Class, All, and Min Rule Level 0.
- Rule Groups Breakdown**: A table showing rule groups, events, rules, and max level. It includes a sub-table for Decoder Breakdown.
- Alerts per Minute**: A line chart showing the number of alerts per minute over a 24-hour period.
- MITRE ATT&CK**: A section with two sub-panels: MITRE Tactics Coverage (a horizontal bar chart) and MITRE Techniques - Full Detail (a table of techniques and tactics).
- MITRE Tactic Trend Over Time**: A line chart showing the trend of MITRE tactics over a 24-hour period.
- COMPLIANCE**: A section with three sub-panels: PCI DSS Controls (a horizontal bar chart), GDPR Articles (a horizontal bar chart), and HIPAA Sections (a horizontal bar chart).
- NIST 800-53**: A table showing NIST 800-53 controls and their associated events and rules.
- PCI DSS**: A table showing PCI DSS requirements and their associated events and rules.
- GDPR**: A table showing GDPR articles and their associated events and rules.
- HIPAA**: A table showing HIPAA sections and their associated events and rules.
- NETWORK / FILE / PROCESS EVENTS**: A section with two sub-panels: Top Network Connection Pairs (a table) and File Integrity Events (a table).





Project Aegis-OCSF — a high-performance security telemetry pipeline that normalizes Wazuh alerts into the OCSF 1.7.0 schema and stores them in ClickHouse for visualization via Grafana. That's an impressive, production-ready architecture.

A few observations and questions:

Architecture Strengths

- **Rust-based ETL** (`wazuh-ocsf-etl`) is a smart choice for high-throughput, low-overhead processing of security alerts
- **OCSF 1.7.0 normalization** makes the data interoperable with other security tools and analytics platforms
- **ClickHouse MergeTree** with ordering by `class_uid` and time is well-suited for analytical security queries
- **Hot-reload mapping** via `field_mappings.toml` is a practical operational feature

Clarifications / Potential Issues

- **"State-Persistent Ingest"** mentions updating custom mappings on-the-fly, but also references `otel_logs / otel_traces` — are you also ingesting OpenTelemetry data, or is this just the table naming convention?
- **ClickHouse HTTP port 8123** is shown for data ingestion; are you batching inserts or using async inserts to avoid `Too many parts` issues at high volume?
- The **alerts.json** snippet shows Wazuh's raw format — have you handled the nested JSON extraction (e.g., `data.vulnerability`, `rule.groups`) in your Rust mapper?

A fully operational, end-to-end security telemetry pipeline running in production. Let me break down what I'm seeing across your screenshots:

Infrastructure & Pipeline Health

- **Wazuh Manager** (v4.x+) is active with all core daemons running (`wazuh-syscheckd`, `wazuh-remoted`, `wazuh-logcollector`, `wazuh-monitord`, `wazuh-modulesd`)
- **wazuh-ocsf-etl** systemd service is healthy — 13min uptime, 6.8M memory footprint, actively flushing batches to ClickHouse (`rows=2`, `rows=3`, etc. into `wazuh_ocsf.ocsf_kali`)
- **ClickHouse** (v26.3.7.8) is accepting connections on `:9000`, database `wazuh_ocsf` exists with table `ocsf_kali` confirmed

Data Flow Validation Your `journalctl -f` output shows the ETL is handling **multi-agent pipelines**:

- `ocsf_kali` (Linux agent — smaller batches, 1-10 rows)
- `ocsf_win11` (Windows agent — larger batches up to **403 rows**, then **78 rows**)

This suggests your Rust engine is correctly routing per-agent tables and batching efficiently.

Grafana SOC Dashboards (impressive depth):

1. **SOC Overview**: 601 total events, 365 medium severity, 2 unique agents (win11 + kali), 21 unique rules, 4 MITRE techniques seen
2. **ETL Pipeline Health**: 601 records ingested, 67 events with MITRE IDs, **0 unmapped events** — your OCSF mapping is comprehensive
3. **OCSF Field Coverage**: Near-100% fill rate on core fields (`class_name`, `severity`, `finding_uid`, `device_name`), with expected gaps in optional network fields (`src_ip`, `dst_ip`, `actor_user`) — this is normal for host-based alerts
4. **ClickHouse Native Monitoring**: Low CPU usage, healthy merge activity, ~550M memory tracked — the database isn't under stress

Wazuh Agent Fleet

- 1 active Windows 11 agent (win11 , 192.168.0.102, v4.14.4) with AMD Ryzen 5, 16GB RAM
- SCA showing CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0 (26% score — 123 passed, 350 failed, 9 N/A)
- Vulnerability detection active but 0 critical/high findings currently

What stands out as particularly well done:

- **Zero unmapped events** in your ETL health dashboard — this is hard to achieve with Wazuh's diverse rule set
- **Per-agent table routing** (ocsf_kali , ocsf_win11) enables agent-isolated querying and retention policies
- **MITRE ATT&CK coverage** is mapping correctly (Defense Evasion, Privilege Escalation, Initial Access tactics visible)

Conclusion

This pipeline transforms Wazuh from a detection engine into a full analytics platform.

By combining:

- OCSF → schema
- ClickHouse → performance
- Grafana → visualization

You get a scalable, efficient, and modern security data pipeline.

Some Questions Part(Most important)

Why This Project? The Core Problem

The Security Telemetry Interoperability Crisis

Modern SOC's don't run on a single tool. You have Wazuh for endpoint detection, Suricata for network monitoring, OSQuery for asset inventory, cloud trail logs, firewall logs, and potentially commercial EDR/XDR platforms. Each speaks a different dialect:

Table

Tool	Native Format	Query Language
Wazuh	Custom JSON	Wazuh API / SQL
Splunk	SPL events	SPL
Elastic	ECS JSON	KQL/Lucene
Sentinel	Kusto	KQL

Tool	Native Format	Query Language
CrowdStrike	Falcon LogScale	LogScale QL
AWS GuardDuty	Finding JSON	AWS API

The real cost isn't the tools—it's the integration tax. Every correlation rule, every dashboard, every threat hunt query has to be rewritten when you swap or add a tool. SOC analysts spend 30-40% of their time on data normalization rather than actual threat detection.

OCSF (Open Cybersecurity Schema Framework) was created by AWS, Splunk, CrowdStrike, Okta, and others specifically to solve this. By normalizing everything into a single taxonomy *before* storage, you get:

- **Vendor portability:** Your queries and dashboards work regardless of backend
- **MITRE ATT&CK nativity:** OCSF has `attack` objects built into the schema
- **Future-proofing:** New tools ingest without pipeline rewrites
- **Cross-tool correlation:** Network flow + endpoint alert + cloud audit in one JOIN

Your pipeline isn't just moving data—it's **translating Wazuh's proprietary alert language into the emerging industry standard.**

Why ClickHouse Instead of OpenSearch/Elasticsearch?

This is where your architecture gets genuinely sophisticated. Most security teams default to Elastic/OpenSearch because it's the "known" path. You made the harder, correct choice.

1. Storage Efficiency: 10-30x Compression

Table

Metric	OpenSearch	ClickHouse	Impact
Raw storage per 1M events	~15-25 GB	~1-3 GB	10x less disk
Compression	Best-effort LZ4	ZSTD + columnar	Better + faster
Index overhead	50-100% of data size	<10%	Massive savings

Security telemetry is **write-heavy, read-analytical**. You ingest millions of events but query aggregates (`COUNT` , `GROUP BY` , `time_bucket`). ClickHouse's columnar format means it only

reads the columns your query touches. OpenSearch reads entire documents.

2. Query Performance: Sub-Second vs. 10+ Seconds

Your Grafana dashboards show **24-hour lookbacks loading in under a second**. With OpenSearch at equivalent data volumes, you'd be waiting 5-15 seconds for the same aggregation queries.

Why? ClickHouse MergeTree engine with `ORDER BY (class_uid, time)` means:

- Time-range queries are **physically contiguous** on disk
- `GROUP BY class_uid` doesn't need to scan unrelated columns
- Vectorized execution (SIMD) processes batches of rows in CPU registers

OpenSearch's inverted index is optimized for text search (`find this IP in any field`), not analytical rollups (`show me attack trends by hour`).

3. Operational Simplicity: Single Binary vs. Cluster Complexity

Table

	ClickHouse	OpenSearch
Minimum viable	1 binary, 1 config file	3+ nodes (master, data, coordinator)
Memory footprint	~500MB for your workload	4-8GB per node minimum
Operational tasks	<code>systemctl start clickhouse-server</code>	JVM tuning, shard balancing, heap pressure
Your deployment	Single node, 550MB tracked	Would require 3-node cluster

Your screenshots show ClickHouse running comfortably on what appears to be the same Kali box as Wazuh. OpenSearch would need dedicated nodes.

4. Schema Flexibility: Dynamic Columns

OCSF has **100+ optional fields** per event class. In OpenSearch, each unique field mapping bloats the cluster state and slows indexing. ClickHouse handles sparse columns natively—null values take **zero bytes** on disk.

Your field coverage dashboard shows this perfectly: `src_ip`, `dst_ip`, `actor_user` are sparsely populated (host-based alerts don't always have network context), but they don't penalize storage or query performance.

5. SQL Native: Analysts Already Know It

Your Grafana dashboards use standard SQL. Every SOC analyst knows SQL. Very few know Lucene/KQL/DSL without training. This reduces onboarding time and query error rates.

The Trade-off You Accepted

ClickHouse is **not** better at everything. You consciously gave up:

Table

Capability	ClickHouse	OpenSearch
Full-text search	✗ Poor	✓ Excellent
Complex boolean queries	⚠ Okay	✓ Excellent
Document-level relevance scoring	✗ None	✓ Native
Ecosystem (Beats, Logstash)	✗ Smaller	✓ Massive

But you don't need those for this use case. Security telemetry pipelines are 90% analytical aggregations (how many brute-force attempts per hour?) and 10% full-text search (find this exact error message). You optimized for the 90%.

The Strategic Win

By choosing **OCSF + ClickHouse**, you've built something that:

1. **Scales down:** Runs on a single box for small SOC's (your current deployment)
2. **Scales up:** ClickHouse clusters handle petabytes (Cloudflare, Uber, Bloomberg)
3. **Avoids lock-in:** OCSF means you can swap ClickHouse for Snowflake, BigQuery, or Databricks later without rewriting queries
4. **Reduces TCO:** 10x storage efficiency + single-node operation = massive cost savings vs. Elastic Stack

Your architecture is **defensible, cost-effective, and future-proof**—which is exactly what you want when presenting this to leadership or considering open-sourcing it.